

Final Architecture Transition Plan for Track-Safe Splits and Codec-Token Renderer Scaffold

Baseline constraints and why this transition needs staging

The core risk you're managing is **evaluation invalidation through leakage** plus **architecture churn** that breaks a known-good baseline. Random row-wise splitting is especially dangerous when your training samples are "segments" that share higher-level structure (here, `track_id`), because segments from the same track can end up on both sides of the split and inflate metrics. A standard remedy is to split by *groups* (tracks) so that all samples belonging to a group land in exactly one partition; this is the same motivation behind group-aware splitters such as `GroupShuffleSplit`, which explicitly splits "according to a third-party provided group" and is controlled by a `random_state` for reproducibility. ¹

From an engineering standpoint, this migration should be treated like a **database schema migration with feature flags**: keep the old "schema" (v1 manifests, v1 checkpoints, v1 model classes, existing CLI entrypoints) as the default, and introduce versioned alternatives behind explicit flags so you can validate each new component in isolation before you rely on it end-to-end.

Two design choices anchor the entire plan:

First, **versioning is opt-in**: v1 remains the default path for model selection, training behavior, manifests, and checkpoint folders, so existing CLI commands remain valid and existing experiment scripts continue to reproduce. Second, **reproducibility is an artifact, not a hope**: any split files, evaluation sample-id lists, and evaluation summaries should be persisted and reused, rather than regenerated implicitly, so later comparisons are scientifically meaningful.

Track-level split enforcement for intent and renderer manifests

The implementation goal is not simply "split by `track_id`," but to do it in a way that is deterministic, reusable, and *provably leak-free*. Group-aware splitting needs to treat `track_id` as the primitive unit, and then re-expand back to segment rows.

A robust approach is:

Read the manifest (intent and renderer separately) and build a mapping from `track_id` → `list(segment_id)` (and any other row keys you need to join back). Construct the set of unique tracks, shuffle deterministically using a seeded RNG, then assign tracks into `train/val/test` using target proportions. Because group splitters' sizes are defined in terms of **groups rather than samples**, you should explicitly report both track counts and segment counts per split to avoid silent distribution shifts (e.g., a few long tracks dominating the segment count in a split). This distinction is explicitly called out in group split APIs: `test_size` and `train_size` refer to groups, not samples. ¹

Persist the split assignment to disk and make it the source of truth. A practical file format is a **track-level assignment file** (one row per `track_id` with a `split` column). This keeps the invariant easy to audit (“one track, one split”), and avoids duplicating millions of segment rows in your split file. When loading a manifest for training/eval, you join the track-split mapping back onto segment rows and filter by split.

You asked for two persisted files—`manifest_intent_splits.csv` and `manifest_renderer_splits.csv`. Keeping them separate is sensible if the two manifests do not cover identical track sets (e.g., audio availability differs), because each module’s evaluation should reflect the data that module can actually see. If you later want cross-module alignment (same held-out tracks for both), that becomes an explicit upgrade (see TODOs).

The validator utility should be a first-class module rather than a one-off script. It should read: (a) the manifest, (b) the split file, then compute and print (and optionally export JSON) the following invariants:

No track leakage: each `track_id` maps to exactly one `split`, and each split’s track IDs are disjoint by construction; the validator should still confirm this (set intersections). This mirrors the core purpose of group-aware splitting: ensure samples from the same group do not appear in both train and test. ¹

No segment leakage: if you also track `segment_id`, ensure each `segment_id` belongs to exactly one split. This is mostly a sanity check against duplicate rows or join bugs.

Counts reported: report number of tracks per split and number of segments per split, plus any key summary stats that matter for evaluation stability (e.g., total duration if available in the manifest). Because group splits are randomized but reproducible via `random_state`, include the seed and a hash of the sorted `(track_id, split)` mapping in the output so you can detect accidental drift. ¹

Determinism should be enforced by a single seed flowing through: split generation, evaluation-subset selection, and any sampling in evaluation exports. The seed should be stored inside the split file header (or a sidecar JSON) and printed by the “preflight” command (covered later).

Unified evaluation harness with reproducible artifacts and optional W&B logging

Your evaluation goals are best met by introducing a single “evaluation runner” abstraction that both intent and renderer training code can call, rather than duplicating logic in two places.

A unified runner should do four things in a fixed order:

Compute metrics for train/val/test for the current model checkpoint and dataset snapshot (or at least val/test every epoch plus train occasionally). For intent, your baseline metrics (MSE, MAE) are straightforward. For thresholded metrics on soft targets (precision/recall/F1), the key engineering decision is to **treat the threshold as configuration** and log it as part of the run metadata so comparisons remain valid.

Export sample predictions for a fixed list of `segment_id`s. This list should live in a committed file (e.g., `eval_samples_segment_ids.txt`) or be logged once and then reused, so every run produces comparable artifacts. Reproducibility here is the same as for splits: don't regenerate implicitly.

Write artifact JSON summaries that make silent failures obvious. At minimum, log tensor shapes, dtypes, min/max/mean/std, number of NaNs/Infs, and a “finite check” boolean per artifact. This pairs well with experiment trackers that store summary statistics for tensors and arrays; for example, W&B's run summary can persist arrays/tensors and store high-level stats such as min/mean/variance/percentiles. ²

Optionally log the above to `entity["company", "Weights & Biases", "ml experiment tracking"]`, while always supporting a local-only mode. W&B logging is naturally step-based: each call to `wandb.Run.log()` advances the step counter by default, and steps drive chart x-axes unless you define a custom axis. ³ If W&B is disabled, your evaluation runner should still write local JSON and media artifacts so the pipeline remains usable without external services.

Because offline/airgapped workflows are common in ML infrastructure, it's worth aligning the local-only mode with W&B's own offline patterns. W&B explicitly supports an offline mode where data is saved locally and later synced. ⁴ Even if you do not rely on `wandb offline` for your non-W&B tracker, designing your directories so that “local artifacts look like something that could be synced” reduces long-term friction.

For renderer evaluation, the harness should support both numerical metrics and qualitative outputs. Even if your baseline loss is MSE on waveform chunks, decoded sample exports are essential for catching regressions that metrics miss. If your renderer v1 already predicts waveform/audio directly, you can export `.wav` from predictions and ground truth. If you later route renderer v2 through tokens, the same evaluation harness can decode predicted tokens back to waveform using the tokenizer's `decode` method. This aligns with codec-token APIs where `encode` and `decode` operate on explicit tensor contracts. ⁵

Renderer v2 scaffold with codec-token interfaces while preserving the conv1d baseline

Your renderer v2 scaffold should be shaped around **tokenizer contracts first**, model second. The reason is that codec-token generation is an interface problem: once you lock down token shapes, time bases, and conditioning semantics, you can iterate on model families (Transformer, Diffusion-on-tokens, hybrid) without rewriting datasets and evaluation.

A credible reference point is EnCodec's role as an audio tokenizer. EnCodec is described as an autoencoder with a residual vector quantization bottleneck that produces several parallel streams of audio tokens with a fixed vocabulary, and those discrete representations are used as inputs for audio language modeling tasks in systems like MusicGen. ⁶ The original EnCodec paper likewise frames it as a neural audio codec with quantized latent space and discusses using lightweight Transformers to further compress the representation, reinforcing the “token interface” framing. ⁷

From a concrete API standpoint, token shapes are not something you should guess—use established codec-token APIs. In AudioCraft's compression model interface, `EncodecModel.encode` returns `codes` with shape `[B, K, T]` where K is the number of codebooks and T is the timestep, and `decode` accepts codes as

an **int tensor of shape [B, K, T]** and returns reconstructed audio [B, C, T]. ⁵ This is exactly the canonical contract you want for your multi-codebook token path.

The MusicGen documentation provides an additional practical anchor: it describes MusicGen as an autoregressive Transformer trained over a 32kHz EnCodec tokenizer “with 4 codebooks sampled at 50 Hz,” generating all 4 codebooks in one pass (with a small delay trick between codebooks). ⁸ That directly supports your need to accommodate both flattened token streams and multi-codebook tensors in your renderer v2 contract.

Given this, a strong scaffold for `renderer/model_v2/` is:

A `tokenizer_interface.py` defining an abstract tokenizer API centered on `encode(audio) → codes, maybe_scale` and `decode(codes, maybe_scale) → audio`, plus properties like `num_codebooks`, `cardinality`, `frame_rate`, and `sample_rate`. Those properties mirror what mature codec interfaces expose (e.g., number of codebooks and frame rate are first-class). ⁵

An `encodec_tokenizer.py` wrapper that integrates an existing EnCodec implementation (or a stub initially). Even in stub mode, the wrapper should define the tensor contracts and return sentinel tensors of correct shapes so the rest of the pipeline can be smoke-tested.

A `renderer_token_transformer.py` skeleton that consumes conditioning (situation + intent) and produces token logits. The key is to establish the “codec-language-model-like” forward signature early: logits shaped either [B, T_tok, V] for single-stream tokens or [B, K, T_tok, V] (or [B, T_tok, K, V]) for multi-codebook outputs.

Your canonical contracts can therefore be formalized as:

Conditioning: `situation` [B, D_s], `intent` [B, T_int, D_int] (as you specified), merged either by cross-attention or by projecting situation into a prefix and concatenating a learned embedding stream.

Token targets: either [B, K, T] as the “native” codec shape or a flattened [B, T] / [B, T, Q] representation if you choose to interleave codebooks or represent them as separate streams. The codec ecosystem’s default is explicit [B, K, T] codes. ⁵

On the data side, the dataset/collate path should be introduced in **optional/stub mode**: if `renderer_model_type=conv1d`, produce waveform chunks exactly as today; if `renderer_model_type=token_transformer`, attempt to produce token targets, but allow a debug mode that returns fake tokens with correct shapes so you can test model initialization, forward pass, and evaluation wiring before doing expensive token caching.

The config flags you listed are well-aligned with this approach:

`renderer_model_type: conv1d | token_transformer` gates the entire model factory.

`renderer_tokenizer_type: wavechunk | encodec` gates whether the renderer output space is waveform chunks (v1) or codec tokens (v2). The term “wavechunk tokenizer” is effectively your current v1 output representation; keeping the naming explicit makes transitions easier to reason about.

Most importantly, the `conv1d + wavechunk` path should remain completely intact and be the default, so your baseline stays runnable.

Intent v2 Transformer planner scaffold behind a flag with v1-compatible outputs

Intent v2 should be scoped similarly: lock down interfaces and safety behavior first, then iterate on modeling. Your requirement that output remains `[B, 60, 7]` is an excellent compatibility constraint: it lets you reuse the v1 renderer conditioning format and v1 losses/metrics while you swap the planner internals.

A Transformer-based planner is justified if you expect richer temporal intent structure than a GRU can easily represent, because Transformers dispense with recurrence and rely solely on attention mechanisms for sequence transduction, enabling parallel processing over timesteps and flexible context aggregation. ⁹ That doesn’t mean it will outperform your GRU immediately, but it’s the standard scaffold for “stronger temporal intent modeling.”

The engineering scaffold should include:

A new `intent/model_v2.py` implementing a Transformer encoder (or encoder-only stack) that ingests situation features plus optional backing features and predicts the same `[B, 60, 7]` tensor. Keep normalization, dropout, positional encoding choices configurable, but avoid proliferating knobs at this stage—this is a scaffold, not a hyperparameter search.

The same losses/metrics as v1 (MSE path remains valid) so that comparisons are apples-to-apples and your evaluation harness can be reused unchanged.

The same hardening behaviors you already rely on in v1: gradient clipping, NaN/Inf checks, and crash diagnostics. For gradient clipping, PyTorch’s `clip_grad_norm_` is the standard utility and supports `error_if_nonfinite` to surface NaNs/Infs promptly rather than silently producing invalid updates. ¹⁰

The config flag `intent_model_type: gru | transformer` should gate model selection in a model factory, with `gru` as default. All CLI paths should resolve to v1 behavior unless explicitly configured otherwise.

For the “one-batch overfit decreases loss” smoke test, the point is not to prove generalization, but to verify that the forward/backward path is wired correctly, the optimizer updates parameters, and numerical guards do not fire spuriously.

Implementation diff and runbook: files, flags, commands, assumptions

The paths below are **proposed** and follow your requested structure (`renderer/model_v2/`, `intent/model_v2.py`) while minimizing edits to stable baseline code by adding new modules and thin factory hooks.

Files changed (full paths)

1) Track-level split enforcement

- `data/splits/track_splitter.py`
- `data/splits/split_validator.py`
- `intent/data/manifest_splits.py` (thin wrapper for intent manifest loading + split join)
- `renderer/data/manifest_splits.py` (thin wrapper for renderer manifest loading + split join)
- `tools/generate_splits.py` (CLI entrypoint or subcommand wiring; should reuse existing CLI patterns)
- `tests/test_track_level_splits.py`
- `tests/test_cli_regression_splits_loading.py`

2) Evaluation + artifacts + standardized logging

- `eval/eval_runner.py`
- `eval/artifacts.py` (JSON summaries, finite checks, audio export helpers)
- `eval/sample_ids.py` (load fixed sample-id list)
- `logging/local_tracker.py` (no-op / JSONL logger when W&B disabled)
- `logging/wandb_tracker.py` (W&B adapter; imported only when enabled)
- `tests/test_eval_runner_local_only.py`
- `tests/test_eval_artifacts_smoke.py`
- `tests/test_renderer_audio_decode_export_smoke.py`

3) Renderer v2 scaffold (codec token path)

- `renderer/model_v2/tokenizer_interface.py`
- `renderer/model_v2/encodec_tokenizer.py`
- `renderer/model_v2/renderer_token_transformer.py`
- `renderer/data/token_targets.py` (dataset/collate optional token target generation; stub-capable)
- `renderer/model_factory.py` (or edit existing factory; minimal diff: add v2 branch)
- `tests/test_tokenizer_interface_shapes.py`
- `tests/test_renderer_token_transformer_forward_shapes.py`
- `tests/test_renderer_v2_init_smoke.py`

4) Intent v2 scaffold (Transformer planner)

- `intent/model_v2.py`
- `intent/model_factory.py` (or minimal edits to existing selection logic)
- `tests/test_intent_transformer_output_shape.py`
- `tests/test_intent_transformer_overfit_smoke.py`
- `tests/test_intent_gru_regression.py`

5) CLI + validation integration

`cli/preflight.py` (new subcommand; prints split integrity, artifact availability, selected versions)
`cli/train_intent.py` (minimal: parse new flags; defaults preserved)
`cli/train_renderer.py` (minimal: parse new flags; defaults preserved)
`scripts/run_eval_suite.sh`

Summary of architecture/version flags added

`data.split_by: segment | track` (default `segment` to preserve v1 unless you are ready to turn it on globally; your goal is `track`)

`data.splits_seed: int` (single seed used for split generation and persisted in split artifacts)

`data.reuse_splits: bool` (default true; if split file exists, reuse)

`data.force_regenerate_splits: bool` (default false; explicit regeneration)

`eval.enabled: bool` (default false unless v1 already evaluates automatically)

`eval.fixed_sample_ids_path: path` (required for artifact reproducibility)

`eval.export_audio: bool` (renderer only; default false)

`logging.tracker: local | wandb` (default `local`; `wandb` opt-in)

`renderer_model_type: conv1d | token_transformer` (default `conv1d`)

`renderer_tokenizer_type: wavechunk | encodec` (default `wavechunk`)

`intent_model_type: gru | transformer` (default `gru`)

Commands to run

Because you required that existing CLI commands must not break, the commands below are expressed as additions that should map onto your current CLI tool (replace `python -m your_cli ...` with your actual entrypoint). None of these should require changing existing invocations.

Split regeneration (intent + renderer)

```
python -m your_cli generate_splits --module intent --split_by track --seed 1337 --force
python -m your_cli generate_splits --module renderer --split_by track --seed 1337 --force
```

```
python -m your_cli preflight --check_splits --module intent
python -m your_cli preflight --check_splits --module renderer
```

Baseline train/eval (v1 unchanged by default)

```
python -m your_cli train_intent --intent_model_type gru --renderer_model_type
conv1d
python -m your_cli train_renderer --renderer_model_type conv1d --
renderer_tokenizer_type wavechunk

python -m your_cli eval --module intent
python -m your_cli eval --module renderer --export_audio
```

V2 scaffold smoke tests (init + forward wiring; no full training required)

```
pytest -q tests/test_tokenizer_interface_shapes.py
pytest -q tests/test_renderer_token_transformer_forward_shapes.py
pytest -q tests/test_renderer_v2_init_smoke.py

pytest -q tests/test_intent_transformer_output_shape.py
pytest -q tests/test_intent_transformer_overfit_smoke.py
```

Full validation suite

```
bash scripts/run_eval_suite.sh
python -m your_cli preflight --check_splits --check_artifacts --
print_selected_versions
```

Assumptions and TODOs left as placeholders

The file paths above assume a Python layout where modules like `intent/`, `renderer/`, `eval/`, and `cli/` exist. If your repository uses `src/` or a different package root, all “full paths” should be translated accordingly.

Manifests are assumed to include `segment_id` and `track_id`. If `track_id` is missing from one manifest, you will need a join key (e.g., an audio-file stem) or a preprocessing step to produce `track_id` deterministically.

Renderer audio export assumes you have a stable waveform representation to write to `.wav`. If renderer v1 predicts a latent that must be postprocessed to audio, the evaluation harness should call the same postprocess function used in inference to avoid mismatched audio.

The EnCodec integration in `encodec_tokenizer.py` is deliberately staged: initial “stub mode” should satisfy contracts and smoke tests, while full codec-token caching or on-the-fly encoding can be introduced later. The codec-token contract should match established implementations where `encode` returns codes shaped `[B, K, T]` and `decode` consumes the same. ⁵

If you want intent and renderer to share an identical held-out test set at the *track* level (stronger end-to-end scientific control), consider adding an optional “shared track split” artifact that both manifests can filter against. This is not required by your current spec, but it can reduce cross-module confounds when comparing full-pipeline outputs.

W&B support should be implemented so that it is safe to disable completely (no import-time dependency failures). When enabled, follow W&B’s step-based logging expectations (`wandb.Run.log()` creates new steps by default) and artifact logging patterns (`Artifact` + `run.log_artifact`). ¹¹

¹ GroupShuffleSplit — scikit-learn 1.8.0 documentation

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupShuffleSplit.html

² Log summary metrics - Weights & Biases Documentation

<https://docs.wandb.ai/models/track/log/log-summary>

³ ¹¹ Overview - Weights & Biases Documentation

<https://docs.wandb.ai/models/track/log>

⁴ wandb offline - Weights & Biases Documentation

<https://docs.wandb.ai/models/ref/cli/wandb-offline>

⁵ audiocraft.models.encodec API documentation

https://facebookresearch.github.io/audiocraft/api_docs/audiocraft/models/encodec.html

⁶ EnCodec

<https://audiocraft.metademolab.com/encodec.html>

⁷ High Fidelity Neural Audio Compression

https://arxiv.org/abs/2210.13438?utm_source=chatgpt.com

⁸ raw.githubusercontent.com

<https://raw.githubusercontent.com/facebookresearch/audiocraft/main/docs/MUSICGEN.md>

⁹ Attention Is All You Need

https://arxiv.org/abs/1706.03762?utm_source=chatgpt.com

¹⁰ torch.nn.utils.clip_grad_norm_ — PyTorch 2.10 ...

https://docs.pytorch.org/docs/stable/generated/torch.nn.utils.clip_grad_norm_.html?utm_source=chatgpt.com